



Red Hat OpenShift AI Self-Managed 2.25

Working with Llama Stack

Working with Llama Stack in Red Hat OpenShift AI Self-Managed

Red Hat OpenShift AI Self-Managed 2.25 Working with Llama Stack

Working with Llama Stack in Red Hat OpenShift AI Self-Managed

Legal Notice

Copyright © 2025 Red Hat, Inc.

The text of and illustrations in this document are licensed by Red Hat under a Creative Commons Attribution–Share Alike 3.0 Unported license ("CC-BY-SA"). An explanation of CC-BY-SA is available at

<http://creativecommons.org/licenses/by-sa/3.0/>

. In accordance with CC-BY-SA, if you distribute this document or an adaptation of it, you must provide the URL for the original version.

Red Hat, as the licensor of this document, waives the right to enforce, and agrees not to assert, Section 4d of CC-BY-SA to the fullest extent permitted by applicable law.

Red Hat, Red Hat Enterprise Linux, the Shadowman logo, the Red Hat logo, JBoss, OpenShift, Fedora, the Infinity logo, and RHCE are trademarks of Red Hat, Inc., registered in the United States and other countries.

Linux[®] is the registered trademark of Linus Torvalds in the United States and other countries.

Java[®] is a registered trademark of Oracle and/or its affiliates.

XFS[®] is a trademark of Silicon Graphics International Corp. or its subsidiaries in the United States and/or other countries.

MySQL[®] is a registered trademark of MySQL AB in the United States, the European Union and other countries.

Node.js[®] is an official trademark of Joyent. Red Hat is not formally related to or endorsed by the official Joyent Node.js open source or commercial project.

The OpenStack[®] Word Mark and OpenStack logo are either registered trademarks/service marks or trademarks/service marks of the OpenStack Foundation, in the United States and other countries and are used with the OpenStack Foundation's permission. We are not affiliated with, endorsed or sponsored by the OpenStack Foundation, or the OpenStack community.

All other trademarks are the property of their respective owners.

Abstract

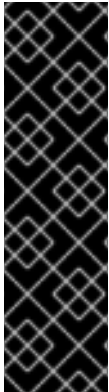
As a cluster administrator, you can use the Llama Stack Operator in Red Hat OpenShift AI.

Table of Contents

CHAPTER 1. OVERVIEW OF LLAMA STACK	3
1.1. THE LLAMASTACKDISTRIBUTION CUSTOM RESOURCE API PROVIDERS	3
1.2. OPENAI COMPATIBILITY FOR RAG APIS IN LLAMA STACK	5
1.3. OPENAI-COMPATIBLE APIS IN LLAMA STACK	6
1.3.1. Supported OpenAI-compatible APIs in OpenShift AI	6
1.3.1.1. Chat Completions API	6
1.3.1.2. Completions API	6
1.3.1.3. Embeddings API	6
1.3.1.4. Files API	6
1.3.1.5. Vector Stores API	7
1.3.1.6. Vector Store Files API	7
1.3.1.7. Models API	7
1.3.1.8. Responses API	7
CHAPTER 2. ACTIVATING THE LLAMA STACK OPERATOR	9
CHAPTER 3. DEPLOYING A RAG STACK IN A DATA SCIENCE PROJECT	11
3.1. OVERVIEW OF RAG	11
3.1.1. Audience for RAG	12
3.2. OVERVIEW OF VECTOR DATABASES	12
3.3. OVERVIEW OF MILVUS VECTOR DATABASES	13
3.4. DEPLOYING A LLAMA MODEL WITH KSERVE	14
3.5. TESTING YOUR VLLM MODEL ENDPOINTS	17
3.6. DEPLOYING A REMOTE MILVUS VECTOR DATABASE	20
3.7. DEPLOYING A LLAMASTACKDISTRIBUTION INSTANCE	24
3.7.1. Example A: LlamaStackDistribution with Inline Milvus	25
3.7.2. Example B: LlamaStackDistribution with Remote Milvus	26
3.8. INGESTING CONTENT INTO A LLAMA MODEL	28
3.9. QUERYING INGESTED CONTENT IN A LLAMA MODEL	31
3.10. PREPARING DOCUMENTS WITH DOCLING FOR LLAMA STACK RETRIEVAL	34
3.11. ABOUT LLAMA STACK SEARCH TYPES	38
3.11.1. Supported search modes	38
3.11.1.1. Keyword search	38
3.11.1.2. Vector search	38
3.11.1.3. Hybrid search	38
3.11.2. Retrieval database support	38
CHAPTER 4. BENCHMARKING EMBEDDING MODELS WITH BEIR DATASETS AND LLAMA STACK	39

CHAPTER 1. OVERVIEW OF LLAMA STACK

Llama Stack is a unified AI runtime environment designed to simplify the deployment and management of generative AI workloads on OpenShift AI. Llama Stack integrates LLM inference servers, vector databases, and retrieval services in a single stack, optimized for Retrieval-Augmented Generation (RAG) and agent-based AI workflows. In OpenShift, the Llama Stack Operator manages the deployment lifecycle of these components, ensuring scalability, consistency, and integration with OpenShift AI projects.



IMPORTANT

Llama Stack integration is currently available in Red Hat OpenShift AI 2.25 as a Technology Preview feature. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).

Llama Stack includes the following components:

- **Inference model servers** such as vLLM, designed to efficiently serve large language models.
- **Vector storage** solutions, primarily Milvus, to store embeddings generated from your domain data.
- **Retrieval and embedding management** workflows using integrated tools, such as Docling, to handle continuous data ingestion and synchronization.
- **Integration with OpenShift AI** by using the **LlamaStackDistribution** custom resource, simplifying configuration and deployment.

For information about how to deploy Llama Stack in OpenShift AI, see [Deploying a RAG stack in a data science project](#).

1.1. THE LLAMASTACKDISTRIBUTION CUSTOM RESOURCE API PROVIDERS

The **LlamaStackDistribution** custom resource includes various API types and providers that you can use in OpenShift AI. The following table displays the supported providers that are included in the distribution:

API type	Providers	How to Enable	Support status
Agents	inline::meta-reference	Enabled by default	Technology Preview
DatasetIO	inline::localfs	Enabled by default	Technology Preview

API type	Providers	How to Enable	Support status
	remote::huggingface	Enabled by default	Technology Preview
Evaluation	remote::trustyai_lmeval	Set the EMBEDDING_MODEL environment variable	Technology Preview
Files	inline::localfs	Enabled by default	Technology Preview
Inference	inline::sentence-transformers	Enabled by default	Technology Preview
	remote::vllm	Set the VLLM_URL environment variable	Technology Preview
	remote::azure	Set the AZURE_API_KEY environment variable	Technology Preview
	remote::bedrock	Set the AWS_ACCESS_KEY_ID environment variable	Technology Preview
	remote::openai	Set the OPENAI_API_KEY environment variable	Technology Preview
	remote::vertexai	Set the VERTEX_AI_PROJECT environment variable	Technology Preview
	remote::watsonx	Set the WATSONX_API_KEY environment variable	Technology Preview
Safety	remote::trustyai_fms	Enabled by default	Technology Preview
Scoring	inline::llm-as-a-judge	Enabled by default	Technology Preview
	inline::basic	Enabled by default	Technology Preview
	inline::braintrust	Enabled by default	Technology Preview
Telemetry	inline::meta-reference	Enabled by default	Technology Preview

API type	Providers	How to Enable	Support status
Tool_runtime	inline::rag-runtime	Enabled by default	Technology Preview
	remote::brave-search	Enabled by default	Technology Preview
	remote::tavily-search	Enabled by default	Technology Preview
	remote::model-context-protocol	Enabled by default	Technology Preview
VectorIO	inline::milvus	Enabled by default	Technology Preview
	remote::milvus	Set the MILVUS_ENDPOINT environment variable	Technology Preview

Additional resources

- [Llama Stack Demos repository](#)
- [Llama Stack Kubernetes Operator documentation](#)
- [Llama Stack documentation](#)

1.2. OPENAI COMPATIBILITY FOR RAG APIS IN LLAMA STACK

OpenShift AI supports OpenAI-compatible request and response schemas for Llama Stack RAG workflows. You can use OpenAI clients and schemas for files, vector stores, and Responses API file search end-to-end.

OpenAI compatibility enables the following capabilities:

- You can use OpenAI SDKs and tools with Llama Stack by setting the client **base_url** to the Llama Stack OpenAI path, **/v1/openai/v1**.
- You can manage files and vector stores by using OpenAI-compatible endpoints. You can then invoke RAG workflows by using the Responses API with the **file_search** tool.

Additional resources

- [OpenAI API Compatibility](#)
- [OpenAI API Reference](#)
- [llama-stack-client-python](#)

1.3. OPENAI-COMPATIBLE APIS IN LLAMA STACK

OpenShift AI includes a Llama Stack component that exposes OpenAI-compatible APIs. These APIs enable you to reuse existing OpenAI SDKs, tools, and workflows directly within your OpenShift environment, without changing your client code. This compatibility layer supports retrieval-augmented generation (RAG), inference, and embedding workloads by using the same endpoints, schemas, and authentication model as OpenAI.

This compatibility layer has the following capabilities:

- **Standardized endpoints:** REST API paths align with OpenAI specifications.
- **Schema parity:** Request and response fields follow OpenAI data structures.



NOTE

When connecting OpenAI SDKs or third-party tools to OpenShift AI, you must update the client configuration to use your deployment's Llama Stack route as the **base_url**. This ensures that API calls are sent to the OpenAI-compatible endpoints that run inside your OpenShift cluster, rather than to the public OpenAI service.

1.3.1. Supported OpenAI-compatible APIs in OpenShift AI

1.3.1.1. Chat Completions API

- **Endpoint:** `/v1/openai/v1/chat/completions`.
- **Providers:** All inference back ends deployed through OpenShift AI.
- **Support level:** Technology Preview.

The Chat Completions API enables conversational, message-based interactions with models served by Llama Stack in OpenShift AI.

1.3.1.2. Completions API

- **Endpoint:** `/v1/openai/v1/completions`.
- **Providers:** All inference backends managed by OpenShift AI.
- **Support level:** Technology Preview.

The Completions API supports single-turn text generation and prompt completion.

1.3.1.3. Embeddings API

- **Endpoint:** `/v1/openai/v1/embeddings`.
- **Providers:** All embedding models enabled in OpenShift AI.

The Embeddings API generates numerical embeddings for text or documents that can be used in downstream semantic search or RAG applications.

1.3.1.4. Files API

- **Endpoint:** `/v1/openai/v1/files`.
- **Providers:** File system-based file storage provider for managing files and documents stored locally in your cluster.
- **Support level:** Technology Preview.

The Files API manages file uploads for use in embedding and retrieval workflows.

1.3.1.5. Vector Stores API

- **Endpoint:** `/v1/openai/v1/vector_stores/`.
- **Providers:** Inline and Remote Milvus configured in OpenShift AI.
- **Support level:** Technology Preview.

The Vector Stores API manages the creation, configuration, and lifecycle of vector store resources in Llama Stack. Through this API, you can create new vector stores, list existing ones, delete unused stores, and query their metadata, all using OpenAI-compatible request and response formats.

1.3.1.6. Vector Store Files API

- **Endpoint:** `/v1/openai/v1/vector_stores/{vector_store_id}/files`.
- **Providers:** Local inline provider configured for file storage and retrieval.
- **Support level:** Developer Preview.

The Vector Store Files API implements the OpenAI Vector Store Files interface and manages the link between document files and Milvus vector stores used for RAG.

1.3.1.7. Models API

- **Endpoint:** `/v1/openai/v1/models`.
- **Providers:** All model-serving back ends configured within OpenShift AI.
- **Support level:** Technology Preview.

The Models API lists and retrieves available model resources from the Llama Stack deployment running on OpenShift AI. By using the Models API, you can enumerate models, view their capabilities, and verify deployment status through a standardized OpenAI-compatible interface.

1.3.1.8. Responses API

- **Endpoint:** `/v1/openai/v1/responses`.
- **Providers:** All inference and retrieval providers configured in OpenShift AI.
- **Support level:** Developer Preview.

The Responses API generates model outputs by combining inference, file search, and tool-calling capabilities through a single OpenAI-compatible endpoint. It is particularly useful for retrieval-augmented generation (RAG) workflows that rely on the **file_search** tool to retrieve context from vector stores.

**NOTE**

The Responses API is an experimental feature that is still under active development in OpenShift AI. While the API is already functional and suitable for evaluation, some endpoints and parameters remain under implementation and might change in future releases. This API is provided for testing and feedback purposes only and is not recommended for production use.

Additional resources

- [OpenAI API Compatibility](#)

CHAPTER 2. ACTIVATING THE LLAMA STACK OPERATOR

You can activate the Llama Stack Operator on your OpenShift cluster by setting its **managementState** to **Managed** in the OpenShift AI Operator **DataScienceCluster** custom resource (CR). This setting enables Llama-based model serving without reinstalling or directly editing Operator subscriptions. You can edit the CR in the OpenShift web console or by using the OpenShift CLI (**oc**).



NOTE

As an alternative to following the steps in this procedure, you can activate the Llama Stack Operator from the OpenShift CLI (**oc**) by running the following command:

```
$ oc patch datasciencecluster <name> --type=merge -p {"spec":{"components":{"llamastackoperator":{"managementState":"Managed"}}}}
```

Replace **<name>** with your **DataScienceCluster** name, for example, **default-dsc**.

Prerequisites

- You have installed OpenShift 4.17 or newer.
- You have cluster administrator privileges.
- You have installed the OpenShift CLI (**oc**) as described in the appropriate documentation for your cluster:
 - [Installing the OpenShift CLI](#) for OpenShift Container Platform
 - [Installing the OpenShift CLI](#) for Red Hat OpenShift Service on AWS
- You have installed the Red Hat OpenShift AI Operator on your cluster.
- You have a **DataScienceCluster** custom resource in your environment; the default is **default-dsc**.
- Your infrastructure supports GPU-enabled instance types, for example, **g4dn.xlarge** on AWS.
- You have enabled GPU support in OpenShift AI, including installing the Node Feature Discovery Operator and NVIDIA GPU Operator. For more information, see [Installing the Node Feature Discovery Operator](#) and [Enabling NVIDIA GPUs](#).
- You have created a **NodeFeatureDiscovery** resource instance on your cluster, as described in [Installing the Node Feature Discovery Operator and creating a NodeFeatureDiscovery instance](#) in the NVIDIA documentation.
- You have created a **ClusterPolicy** resource instance with default values on your cluster, as described in [Creating the ClusterPolicy instance](#) in the NVIDIA documentation.

Procedure

1. Log in to the OpenShift web console as a cluster administrator.
2. In the **Administrator** perspective, click **Operators** → **Installed Operators**.
3. Click the **Red Hat OpenShift AI Operator** to open its details.

4. Click the **Data Science Cluster** tab.
5. On the **DataScienceClusters** page, click the **default-dsc** object.
6. Click the **YAML** tab.
An embedded YAML editor opens, displaying the configuration for the **DataScienceCluster** custom resource.
7. In the YAML editor, locate the **spec.components** section. If the **llamastackoperator** field does not exist, add it. Then, set the **managementState** field to **Managed**:

```
spec:
  components:
    llamastackoperator:
      managementState: Managed
```

8. Click **Save** to apply your changes.

Verification

After you activate the Llama Stack Operator, verify that it is running in your cluster:

1. In the OpenShift web console, click **Workloads → Pods**.
2. From the **Project** list, select the **redhat-ods-applications** namespace.
3. Confirm that a pod with the label **app.kubernetes.io/name=llama-stack-operator** is displayed and has a status of **Running**.

CHAPTER 3. DEPLOYING A RAG STACK IN A DATA SCIENCE PROJECT



IMPORTANT

This feature is currently available in Red Hat OpenShift AI 2.25 as a Technology Preview feature. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).

As an OpenShift cluster administrator, you can deploy a Retrieval-Augmented Generation (RAG) stack in OpenShift AI. This stack provides the infrastructure, including LLM inference, vector storage, and retrieval services that data scientists and AI engineers use to build conversational workflows in their projects.

To deploy the RAG stack in a data science project, complete the following tasks:

- Activate the Llama Stack Operator in OpenShift AI.
- Enable GPU support on the OpenShift cluster. This task includes installing the required NVIDIA Operators.
- Deploy an inference model, for example, the llama-3.2-3b-instruct model. This task includes creating a storage connection and configuring GPU allocation.
- Create a **LlamaStackDistribution** instance to enable RAG functionality. This action deploys LlamaStack alongside a Milvus vector store and connects both components to the inference model.
- Ingest domain data into Milvus by running Docling in a data science pipeline or Jupyter notebook. This process keeps the embeddings synchronized with the source data.
- Expose and secure the model endpoints.

3.1. OVERVIEW OF RAG

Retrieval-augmented generation (RAG) in OpenShift AI enhances large language models (LLMs) by integrating domain-specific data sources directly into the model's context. Domain-specific data sources can be structured data, such as relational database tables, or unstructured data, such as PDF documents.

RAG indexes content and builds an embedding store that data scientists and AI engineers can query. When data scientists or AI engineers pose a question to a RAG chatbot, the RAG pipeline retrieves the most relevant pieces of data, passes them to the LLM as context, and generates a response that reflects both the prompt and the retrieved content.

By implementing RAG, data scientists and AI engineers can obtain tailored, accurate, and verifiable answers to complex queries based on their own datasets within a data science project.

3.1.1. Audience for RAG

The target audience for RAG is practitioners who build data-grounded conversational AI applications using OpenShift AI infrastructure.

For Data Scientists

Data scientists can use RAG to prototype and validate models that answer natural-language queries against data sources without managing low-level embedding pipelines or vector stores. They can focus on creating prompts and evaluating model outputs instead of building retrieval infrastructure.

For MLOps Engineers

MLOps engineers typically deploy and operate RAG pipelines in production. Within OpenShift AI, they manage LLM endpoints, monitor performance, and ensure that both retrieval and generation scale reliably. RAG decouples vector store maintenance from the serving layer, enabling MLOps engineers to apply CI/CD workflows to data ingestion and model deployment alike.

For Data Engineers

Data engineers build workflows to load data into storage that OpenShift AI indexes. They keep embeddings in sync with source systems, such as S3 buckets or relational tables to ensure that chatbot responses are accurate.

For AI Engineers

AI engineers architect RAG chatbots by defining prompt templates, retrieval methods, and fallback logic. They configure agents and add domain-specific tools, such as OpenShift job triggers, enabling rapid iteration.

3.2. OVERVIEW OF VECTOR DATABASES

Vector databases are a crucial component of retrieval-augmented generation (RAG) in OpenShift AI. They store and index vector embeddings that represent the semantic meaning of text or other data. When you integrate vector databases with Llama Stack in OpenShift AI, you can build RAG applications that combine large language models (LLMs) with relevant, domain-specific knowledge.

Vector databases provide you with the following capabilities:

- Store vector embeddings generated by embedding models.
- Support efficient similarity search to retrieve semantically related content.
- Enable RAG workflows by supplying the LLM with contextually relevant data from a specific domain.

When you deploy RAG workloads in OpenShift AI, you can deploy vector databases through the Llama Stack Operator. Currently, OpenShift AI supports the following vector databases:

- **Inline Milvus Lite** An Inline Milvus vector database runs embedded within the Llama Stack Distribution (LSD) pod and is suitable for lightweight experimentation and small-scale development. Inline Milvus stores data in a local SQLite database and is limited in scale and persistence.
- **Remote Milvus** A remote Milvus vector database runs as a standalone service in your project namespace or as an external managed deployment. Remote Milvus is recommended for production-grade RAG use cases because it provides persistence, scalability, and isolation from the Llama Stack Distribution (LSD) pod. In OpenShift environments, you must deploy Milvus with an etcd service directly in your project. For more information on using etcd services, see [Providing redundancy with etcd](#).

Consider the following points when you decide on the vector database to use for your RAG workloads:

- Use **inline Milvus Lite** if you want to experiment quickly with RAG in a self-contained setup and do not require persistence across pod restarts.
- Use **remote Milvus** if you need reliable storage, high availability, and the ability to scale out RAG workloads in your OpenShift AI environment.

3.3. OVERVIEW OF MILVUS VECTOR DATABASES

Milvus is an open source vector database designed for high-performance similarity search across embedding data. In OpenShift AI, Milvus is supported as a remote vector database provider for the Llama Stack Operator. Milvus enables retrieval-augmented generation (RAG) workloads that require persistence, scalability, and efficient search across large document collections.

Milvus vector databases provide you with the following capabilities in OpenShift AI:

- Similarity search using Approximate Nearest Neighbor (ANN) algorithms.
- Persistent storage support for vectors.
- Indexing and query optimizations for embedding-based search.
- Integration with external metadata and APIs.

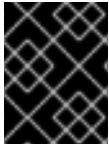
In OpenShift AI, you can use Milvus vector databases in the following operational modes:

- **Inline Milvus Lite**, which runs embedded in the Llama Stack Distribution pod for testing or small-scale experiments.
- **Remote Milvus**, which runs as a standalone service in your OpenShift project or as an external managed Milvus service. Remote Milvus is recommended for production workloads.

When you deploy a remote Milvus vector database, you must run the following components in your OpenShift project:

- **Secret (milvus-secret)**: Stores sensitive data such as the Milvus root password.
- **PersistentVolumeClaim (milvus-pvc)**: Provides persistent storage for Milvus data.
- **Deployment (etcd-deployment)**: Runs an etcd instance that Milvus uses for metadata storage and service coordination.
- **Service (etcd-service)**: Exposes the etcd port for Milvus to connect to.
- **Deployment (milvus-standalone)**: Runs Milvus in standalone mode and connects it to the etcd service and PVC.
- **Service (milvus-service)**: Exposes Milvus gRPC (19530) and HTTP (9091 health check) ports for client access.

Milvus requires an etcd service to manage metadata such as collections, indexes, and partitions, and to provide service discovery and coordination among Milvus components. Even when running in standalone mode, Milvus depends on etcd to operate correctly and maintain metadata consistency. For more information on using etcd services, see [Providing redundancy with etcd](#).



IMPORTANT

Do not use the OpenShift control plane etcd for Milvus. You must deploy a separate etcd instance inside your project or connect to an external etcd service.

Use Remote Milvus when you require a persistent, scalable, and production-ready vector database that integrates seamlessly with OpenShift AI. Consider choosing a remote Milvus vector database if your deployment must cater for the following requirements:

- Persistent vector storage across restarts or upgrades.
- Scalable indexing and high-performance vector search.
- A production-grade RAG architecture integrated with OpenShift AI.

3.4. DEPLOYING A LLAMA MODEL WITH KSERVE

To use Llama Stack and retrieval-augmented generation (RAG) workloads in OpenShift AI, you must deploy a Llama model with a vLLM model server and configure KServe in KServe RawDeployment mode.

Prerequisites

- You have installed OpenShift 4.17 or newer.
- You have logged in to Red Hat OpenShift AI.
- You have cluster administrator privileges for your OpenShift cluster.
- You have activated the Llama Stack Operator.
- You have installed KServe.
- You have enabled the single-model serving platform. For more information about enabling the single-model serving platform, see [Enabling the single-model serving platform](#).
- You can access the single-model serving platform in the dashboard configuration. For more information about setting dashboard configuration options, see [Customizing the dashboard](#).
- You have enabled GPU support in OpenShift AI, including installing the Node Feature Discovery Operator and NVIDIA GPU Operator. For more information, see [Installing the Node Feature Discovery Operator](#) and [Enabling NVIDIA GPUs](#).
- You have installed the OpenShift CLI (**oc**) as described in the appropriate documentation for your cluster:
 - [Installing the OpenShift CLI](#) for OpenShift Container Platform
 - [Installing the OpenShift CLI](#) for Red Hat OpenShift Service on AWS
- You have created a data science project.
- The vLLM serving runtime is installed and available in your environment.
- You have created a storage connection for your model that contains a **URI - v1** connection type. This storage connection must define the location of your Llama 3.2 model artifacts. For

example, `oci://quay.io/redhat-ai-services/modelcar-catalog:llama-3.2-3b-instruct`. For more information about creating storage connections, see [Adding a connection to your data science project](#).



PROCEDURE

These steps are only supported in OpenShift AI versions 2.19 and later.

1. In the OpenShift AI dashboard, navigate to the project details page and click the **Models** tab.
2. In the **Single-model serving platform** tile, click **Select single-model**.
3. Click the **Deploy model** button.
The **Deploy model** dialog opens.
4. Configure the deployment properties for your model:
 - a. In the **Model deployment name** field, enter a unique name for your deployment.
 - b. In the **Serving runtime** field, select **vLLM NVIDIA GPU serving runtime for KServe** from the drop-down list.
 - c. In the **Deployment mode** field, select **KServe RawDeployment** from the drop-down list.
 - d. Set **Number of model server replicas to deploy** to **1**.
 - e. In the **Model server size** field, select **Custom** from the drop-down list.
 - Set **CPUs requested** to **1 core**.
 - Set **Memory requested** to **10 GiB**.
 - Set **CPU limit** to **2 core**.
 - Set **Memory limit** to **14 GiB**.
 - Set **Accelerator** to **NVIDIA GPUs**.
 - Set **Accelerator count** to **1**.
 - f. From the **Connection type**, select a relevant data connection from the drop-down list.
5. In the **Additional serving runtime arguments** field, specify the following recommended arguments:

```
--dtype=half
--max-model-len=20000
--gpu-memory-utilization=0.95
--enable-chunked-prefill
--enable-auto-tool-choice
--tool-call-parser=llama3_json
--chat-template=/app/data/template/tool_chat_template_llama3.2_json.jinja
```

- a. Click **Deploy**.

**NOTE**

Model deployment can take several minutes, especially for the first model that is deployed on the cluster. Initial deployment may take more than 10 minutes while the relevant images download.

Verification

1. Verify that the **kserve-controller-manager** and **odh-model-controller** pods are running:
 - a. Open a new terminal window.
 - b. Log in to your OpenShift cluster from the CLI:
 - c. In the upper-right corner of the OpenShift web console, click your user name and select **Copy login command**.
 - d. After you have logged in, click **Display token**.
 - e. Copy the **Log in with this token** command and paste it in the OpenShift CLI (**oc**).

```
$ oc login --token=<token> --server=<openshift_cluster_url>
```

- f. Enter the following command to verify that the **kserve-controller-manager** and **odh-model-controller** pods are running:

```
$ oc get pods -n redhat-ods-applications | grep -E 'kserve-controller-manager|odh-model-controller'
```

- g. Confirm that you see output similar to the following example:

```
kserve-controller-manager-7c865c9c9f-xyz12 1/1 Running 0 4m21s
odh-model-controller-7b7d5fd9cc-wxy34 1/1 Running 0 3m55s
```

- h. If you do not see either of the **kserve-controller-manager** and **odh-model-controller** pods, there could be a problem with your deployment. In addition, if the pods appear in the list, but their **Status** is not set to **Running**, check the pod logs for errors:

```
$ oc logs <pod-name> -n redhat-ods-applications
```

- i. Check the status of the inference service:

```
$ oc get inferencingservice -n llamastack
$ oc get pods -n <data science project name> | grep llama
```

- The deployment automatically creates the following resources:
 - A **ServingRuntime** resource.
 - An **InferenceService** resource, a **Deployment**, a pod, and a service pointing to the pod.
- Verify that the server is running. For example:

```
$ oc logs llama-32-3b-instruct-predictor-77f6574f76-8nl4r -n <data science project name>
```

Check for output similar to the following example log:

```
INFO 2025-05-15 11:23:52,750 __main__:498 server: Listening on ['::', '0.0.0.0']:8321
INFO: Started server process [1]
INFO: Waiting for application startup.
INFO 2025-05-15 11:23:52,765 __main__:151 server: Starting up
INFO: Application startup complete.
INFO: Uvicorn running on http://['::', '0.0.0.0']:8321 (Press CTRL+C to quit)
```

- The deployed model displays in the **Models** tab on the Data Science project details page for the project it was deployed under.
2. If you see a **ConvertTritonGPToLLVM** error in the pod logs when querying the **/v1/chat/completions** API, and the vLLM server restarts or returns a **500 Internal Server** error, apply the following workaround:
- Before deploying the model, remove the **--enable-chunked-prefill** argument from the **Additional serving runtime arguments** field in the deployment dialog.

The error is displayed similar to the following:

```
/opt/vllm/lib64/python3.12/site-packages/vllm/attention/ops/prefix_prefill.py:36:0: error:
Failures have been detected while processing an MLIR pass pipeline
/opt/vllm/lib64/python3.12/site-packages/vllm/attention/ops/prefix_prefill.py:36:0: note:
Pipeline failed while executing ['ConvertTritonGPToLLVM' on 'builtin.module' operation]:
reproducer generated at `std::errs, please share the reproducer above with Triton project.`
INFO: 10.129.2.8:0 - "POST /v1/chat/completions HTTP/1.1" 500 Internal Server Error
```

3.5. TESTING YOUR VLLM MODEL ENDPOINTS

To verify that your deployed Llama 3.2 model is accessible externally, ensure that your vLLM model server is exposed as a network endpoint. You can then test access to the model from outside both the OpenShift cluster and the OpenShift AI interface.



IMPORTANT

If you selected **Make deployed models available through an external route** during deployment, your vLLM model endpoint is already accessible outside the cluster. You do not need to manually expose the model server. Manually exposing vLLM model endpoints, for example, by using **oc expose**, creates an unsecured route unless you configure authentication. Avoid exposing endpoints without security controls to prevent unauthorized access.

Prerequisites

- You have cluster administrator privileges for your OpenShift cluster.
- You have logged in to Red Hat OpenShift AI.
- You have activated the Llama Stack Operator in OpenShift AI.

- You have deployed an inference model, for example, the llama-3.2-3b-instruct model.
- You have installed the OpenShift CLI (**oc**) as described in the appropriate documentation for your cluster:
 - [Installing the OpenShift CLI](#) for OpenShift Container Platform
 - [Installing the OpenShift CLI](#) for Red Hat OpenShift Service on AWS

Procedure

1. Open a new terminal window.
 - a. Log in to your OpenShift cluster from the CLI:
 - b. In the upper-right corner of the OpenShift web console, click your user name and select **Copy login command**.
 - c. After you have logged in, click **Display token**.
 - d. Copy the **Log in with this token** command and paste it in the OpenShift CLI (**oc**).

```
$ oc login --token=<token> --server=<openshift_cluster_url>
```

2. If you enabled **Require token authentication** during model deployment, retrieve your token:

```
$ export MODEL_TOKEN=$(oc get secret default-name-llama-32-3b-instruct-sa -n <project name> --template={{ .data.token }} | base64 -d)
```

3. Obtain your model endpoint URL:

- If you enabled **Make deployed models available through an external route** during model deployment, click **Endpoint details** on the **Model deployments** page in the OpenShift AI dashboard to obtain your model endpoint URL.
- In addition, if you did not enable **Require token authentication** during model deployment, you can also enter the following command to retrieve the endpoint URL:

```
$ export MODEL_ENDPOINT="https://$(oc get route llama-32-3b-instruct -n <project name> --template={{ .spec.host }})"
```

4. Test the endpoint with a sample chat completion request:

- If you did not enable **Require token authentication** during model deployment, enter a chat completion request. For example:

```
$ curl -X POST $MODEL_ENDPOINT/v1/chat/completions \
-H "Content-Type: application/json" \
-d '{
  "model": "llama-32-3b-instruct",
  "messages": [
    {
      "role": "user",
      "content": "Hello"
```

```
}
]
}'
```

- If you enabled **Require token authentication** during model deployment, include a token in your request. For example:

```
curl -s -k $MODEL_ENDPOINT/v1/chat/completions \
--header "Authorization: Bearer $MODEL_TOKEN" \
--header 'Content-Type: application/json' \
-d '{
  "model": "llama-32-3b-instruct",
  "messages": [
    {
      "role": "user",
      "content": "can you tell me a funny joke?"
    }
  ]
}' | jq .
```



NOTE

The **-k** flag disables SSL verification and should only be used in test environments or with self-signed certificates.

Verification

Confirm that you received a JSON response containing a chat completion. For example:

```
{
  "id": "chatcmpl-05d24b91b08a4b78b0e084d4cc91dd7e",
  "object": "chat.completion",
  "created": 1747279170,
  "model": "llama-32-3b-instruct",
  "choices": [{
    "index": 0,
    "message": {
      "role": "assistant",
      "reasoning_content": null,
      "content": "Hello! It's nice to meet you. Is there something I can help you with or would you like to chat?",
      "tool_calls": []
    },
    "logprobs": null,
    "finish_reason": "stop",
    "stop_reason": null
  }],
  "usage": {
    "prompt_tokens": 37,
    "total_tokens": 62,
    "completion_tokens": 25,
    "prompt_tokens_details": null
  },
  "prompt_logprobs": null
}
```

If you do not receive a response similar to the example, verify that the endpoint URL and token are correct, and ensure your model deployment is running.

3.6. DEPLOYING A REMOTE MILVUS VECTOR DATABASE

To use Milvus as a remote vector database provider for Llama Stack in OpenShift AI, you must deploy Milvus and its required etcd service in your OpenShift project. This procedure shows how to deploy Milvus in standalone mode without the Milvus Operator.



NOTE

The following example configuration is intended for testing or evaluation environments. For production-grade deployments, see <https://milvus.io/docs> in the Milvus documentation.

Prerequisites

- You have installed OpenShift 4.17 or newer.
- You have enabled GPU support in OpenShift AI. This includes installing the Node Feature Discovery operator and NVIDIA GPU Operators. For more information, see [Installing the Node Feature Discovery operator](#) and [Enabling NVIDIA GPUs](#).
- You have cluster administrator privileges for your OpenShift cluster.
- You are logged in to Red Hat OpenShift AI.
- You have a StorageClass available that can provision persistent volumes.
- You created a root password to secure your Milvus service.
- You have deployed an inference model with vLLM, for example, the llama-3.2-3b-instruct model, and you have selected **Make deployed models available through an external route** and **Require token authentication** during model deployment.
- You have the correct inference model identifier, for example, llama-3-2-3b.
- You have the model endpoint URL, ending with **/v1**, such as **https://llama-32-3b-instruct-predictor:8443/v1**.
- You have the API token required to access the model endpoint.
- You have installed the OpenShift command line interface (**oc**) as described in [Installing the OpenShift CLI](#).

Procedure

1. In the OpenShift console, click the **Quick Create** () icon and then click the **Import YAML** option.
2. Verify that your data science project is the selected project.
3. In the **Import YAML** editor, paste the following manifest and click **Create**:


```

apiVersion: v1
kind: Secret
metadata:
  name: milvus-secret
type: Opaque
stringData:
  root-password: "MyStr0ngP@ssw0rd"
---
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: milvus-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 20Gi
  volumeMode: Filesystem
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: etcd-deployment
  labels:
    app: etcd
spec:
  replicas: 1
  selector:
    matchLabels:
      app: etcd
  strategy:
    type: Recreate
  template:
    metadata:
      labels:
        app: etcd
    spec:
      containers:
        - name: etcd
          image: quay.io/coreos/etcd:v3.5.5
          command:
            - etcd
            - --advertise-client-urls=http://127.0.0.1:2379
            - --listen-client-urls=http://0.0.0.0:2379
            - --data-dir=/etcd
          ports:
            - containerPort: 2379
      volumeMounts:
        - name: etcd-data
          mountPath: /etcd
      env:
        - name: ETCD_AUTO_COMPACTION_MODE
          value: revision
        - name: ETCD_AUTO_COMPACTION_RETENTION
          value: "1000"

```

```

      - name: ETCD_QUOTA_BACKEND_BYTES
        value: "4294967296"
      - name: ETCD_SNAPSHOT_COUNT
        value: "50000"
    volumes:
      - name: etcd-data
        emptyDir: {}
    restartPolicy: Always
  ---
apiVersion: v1
kind: Service
metadata:
  name: etcd-service
spec:
  ports:
    - port: 2379
      targetPort: 2379
  selector:
    app: etcd
  ---
apiVersion: apps/v1
kind: Deployment
metadata:
  labels:
    app: milvus-standalone
  name: milvus-standalone
spec:
  replicas: 1
  selector:
    matchLabels:
      app: milvus-standalone
  strategy:
    type: Recreate
  template:
    metadata:
      labels:
        app: milvus-standalone
    spec:
      containers:
        - name: milvus-standalone
          image: milvusdb/milvus:v2.6.0
          args: ["milvus", "run", "standalone"]
          env:
            - name: DEPLOY_MODE
              value: standalone
            - name: ETCD_ENDPOINTS
              value: etcd-service:2379
            - name: COMMON_STORAGETYPE
              value: local
            - name: MILVUS_ROOT_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: milvus-secret
                  key: root-password
          livenessProbe:
            exec:

```

```

    command: ["curl", "-f", "http://localhost:9091/healthz"]
    initialDelaySeconds: 90
    periodSeconds: 30
    timeoutSeconds: 20
    failureThreshold: 5
  ports:
    - containerPort: 19530
      protocol: TCP
    - containerPort: 9091
      protocol: TCP
  volumeMounts:
    - name: milvus-data
      mountPath: /var/lib/milvus
  restartPolicy: Always
  volumes:
    - name: milvus-data
      persistentVolumeClaim:
        claimName: milvus-pvc
---
apiVersion: v1
kind: Service
metadata:
  name: milvus-service
spec:
  selector:
    app: milvus-standalone
  ports:
    - name: grpc
      port: 19530
      targetPort: 19530
    - name: http
      port: 9091
      targetPort: 9091

```



NOTE

- Use the gRPC port (**19530**) for the **MILVUS_ENDPOINT** setting in Llama Stack.
- The HTTP port (**9091**) is reserved for health checks.
- If you deploy Milvus in a different namespace, use the fully qualified service name in your Llama Stack configuration. For example: **http://milvus-service.<namespace>.svc.cluster.local:19530**

Verification

1. In the OpenShift web console, click **Workloads → Deployments**.
2. Verify that both **etcd-deployment** and **milvus-standalone** show a status of **1 of 1 pods available**.
3. Click **Pods** in the navigation panel and confirm that pods for both deployments are **Running**.
4. Click the **milvus-standalone** pod name, then select the **Logs** tab.

5. Verify that Milvus reports a healthy startup with output similar to:

```
Milvus Standalone is ready to serve ...
Listening on 0.0.0.0:19530 (gRPC)
```

6. Click **Networking** → **Services** and confirm that the **milvus-service** and **etcd-service** resources exist and are exposed on ports **19530** and **2379**, respectively.
7. (Optional) Click **Pods** → **milvus-standalone** → **Terminal** and run the following health check:

```
curl http://localhost:9091/healthz
```

A response of `{"status": "healthy"}` confirms that Milvus is running correctly.

3.7. DEPLOYING A LLAMASTACKDISTRIBUTION INSTANCE

You can deploy Llama Stack with retrieval-augmented generation (RAG) by pairing it with a vLLM-served Llama 3.2 model. This module provides two deployment examples of the **LlamaStackDistribution** custom resource (CR): one configured for Inline Milvus (single-node, embedded) and one for Remote Milvus (external Milvus service). When you create the CR, specify **rh-dev** in the **spec.server.distribution.name** field.

Prerequisites

- You have installed OpenShift 4.17 or newer.
- You have enabled GPU support in OpenShift AI. This includes installing the Node Feature Discovery Operator and NVIDIA GPU Operator. For more information, see [Installing the Node Feature Discovery Operator](#) and [Enabling NVIDIA GPUs](#).
- You have cluster administrator privileges for your OpenShift cluster.
- You are logged in to Red Hat OpenShift AI.
- You have activated the Llama Stack Operator in OpenShift AI.
- You have deployed an inference model with vLLM (for example, **llama-3.2-3b-instruct**) and selected **Make deployed models available through an external route** and **Require token authentication** during model deployment.
- You have the correct inference model identifier, for example, **llama-3-2-3b**.
- You have the model endpoint URL ending with **/v1**, for example, **https://llama-32-3b-instruct-predictor:8443/v1**.
- You have the API token required to access the model endpoint.
- You have installed the OpenShift CLI (**oc**) as described in the appropriate documentation for your cluster:
 - [Installing the OpenShift CLI](#) for OpenShift Container Platform
 - [Installing the OpenShift CLI](#) for Red Hat OpenShift Service on AWS

Procedure

1. Open a new terminal window and log in to your OpenShift cluster from the CLI:
In the upper-right corner of the OpenShift web console, click your user name and select **Copy login command**. After you have logged in, click **Display token**. Copy the **Log in with this token** command and paste it in the OpenShift CLI (**oc**).

```
$ oc login --token=<token> --server=<openshift_cluster_url>
```

2. Create a secret that contains the inference model environment variables:

```
export INFERENCE_MODEL="llama-3-2-3b"
export VLLM_URL="https://llama-32-3b-instruct-predictor:8443/v1"
export VLLM_TLS_VERIFY="false" # Use "true" in production
export VLLM_API_TOKEN="<token identifier>"

oc create secret generic llama-stack-inference-model-secret \
  --from-literal=INFERENCE_MODEL="$INFERENCE_MODEL" \
  --from-literal=VLLM_URL="$VLLM_URL" \
  --from-literal=VLLM_TLS_VERIFY="$VLLM_TLS_VERIFY" \
  --from-literal=VLLM_API_TOKEN="$VLLM_API_TOKEN"
```

3. Choose **one** of the following deployment examples:

3.7.1. Example A: LlamaStackDistribution with Inline Milvus

Use this example for development or small datasets where an embedded, single-node Milvus is sufficient. No **MILVUS_*** connection variables are required.

1. In the OpenShift web console, select **Administrator** → **Quick Create** () → **Import YAML**, and create a CR similar to the following:

```
apiVersion: llamastack.io/v1alpha1
kind: LlamaStackDistribution
metadata:
  name: lsd-llama-milvus-inline
spec:
  replicas: 1
  server:
    containerSpec:
      resources:
        requests:
          cpu: "250m"
          memory: "500Mi"
        limits:
          cpu: 4
          memory: "12Gi"
    env:
      - name: INFERENCE_MODEL
        valueFrom:
          secretKeyRef:
            name: llama-stack-inference-model-secret
            key: INFERENCE_MODEL
      - name: VLLM_MAX_TOKENS
```

```

    value: "4096"
  - name: VLLM_URL
    valueFrom:
      secretKeyRef:
        name: llama-stack-inference-model-secret
        key: VLLM_URL
  - name: VLLM_TLS_VERIFY
    valueFrom:
      secretKeyRef:
        name: llama-stack-inference-model-secret
        key: VLLM_TLS_VERIFY
  - name: VLLM_API_TOKEN
    valueFrom:
      secretKeyRef:
        name: llama-stack-inference-model-secret
        key: VLLM_API_TOKEN
name: llama-stack
port: 8321
distribution:
  name: rh-dev

```



NOTE

The **rh-dev** value is an internal image reference. When you create the **LlamaStackDistribution** custom resource, the OpenShift AI Operator automatically resolves **rh-dev** to the container image in the appropriate registry. This internal image reference allows the underlying image to update without requiring changes to your custom resource.

3.7.2. Example B: LlamaStackDistribution with Remote Milvus

Use this example for production-grade or large datasets with an external Milvus service. This configuration reads both **MILVUS_ENDPOINT** and **MILVUS_TOKEN** from a dedicated secret.

1. Create the Milvus connection secret:

```

# Required: gRPC endpoint on port 19530
export MILVUS_ENDPOINT="tcp://milvus-service:19530"
export MILVUS_TOKEN="<milvus-root-or-user-token>"
export MILVUS_CONSISTENCY_LEVEL="Bounded" # Optional; choose per your
deployment

oc create secret generic milvus-secret \
  --from-literal=MILVUS_ENDPOINT="$MILVUS_ENDPOINT" \
  --from-literal=MILVUS_TOKEN="$MILVUS_TOKEN" \
  --from-literal=MILVUS_CONSISTENCY_LEVEL="$MILVUS_CONSISTENCY_LEVEL"

```



IMPORTANT

Use the **gRPC port 19530** for **MILVUS_ENDPOINT**. Ports such as **9091** are typically used for health checks and are not valid for client traffic.

2. In the OpenShift web console, select **Administrator** → **Quick Create** () → **Import YAML**, and create a CR similar to the following:

```

apiVersion: llamastack.io/v1alpha1
kind: LlamaStackDistribution
metadata:
  name: lsdl-llama-milvus-remote
spec:
  replicas: 1
  server:
    containerSpec:
      resources:
        requests:
          cpu: "250m"
          memory: "500Mi"
        limits:
          cpu: 4
          memory: "12Gi"
      env:
        - name: INFERENCE_MODEL
          valueFrom:
            secretKeyRef:
              name: llama-stack-inference-model-secret
              key: INFERENCE_MODEL
        - name: VLLM_MAX_TOKENS
          value: "4096"
        - name: VLLM_URL
          valueFrom:
            secretKeyRef:
              name: llama-stack-inference-model-secret
              key: VLLM_URL
        - name: VLLM_TLS_VERIFY
          valueFrom:
            secretKeyRef:
              name: llama-stack-inference-model-secret
              key: VLLM_TLS_VERIFY
        - name: VLLM_API_TOKEN
          valueFrom:
            secretKeyRef:
              name: llama-stack-inference-model-secret
              key: VLLM_API_TOKEN
        # --- Remote Milvus configuration from secret ---
        - name: MILVUS_ENDPOINT
          valueFrom:
            secretKeyRef:
              name: milvus-secret
              key: MILVUS_ENDPOINT
        - name: MILVUS_TOKEN
          valueFrom:
            secretKeyRef:
              name: milvus-secret
              key: MILVUS_TOKEN
        - name: MILVUS_CONSISTENCY_LEVEL
          valueFrom:
            secretKeyRef:

```

```

name: milvus-secret
key: MILVUS_CONSISTENCY_LEVEL
name: llama-stack
port: 8321
distribution:
name: rh-dev

```

3. Click **Create**.

Verification

- In the left-hand navigation, click **Workloads → Pods** and verify that the Llama Stack pod is running in the correct namespace.
- To verify that the Llama Stack server is running, click the pod name and select the **Logs** tab. Look for output similar to the following:

```

INFO    2025-05-15 11:23:52,750 __main__:498 server: Listening on [':', '0.0.0.0']:8321
INFO:   Started server process [1]
INFO:   Waiting for application startup.
INFO    2025-05-15 11:23:52,765 __main__:151 server: Starting up
INFO:   Application startup complete.
INFO:   Uvicorn running on http://[':', '0.0.0.0']:8321 (Press CTRL+C to quit)

```

- Confirm that a Service resource for the Llama Stack backend is present in your namespace and points to the running pod: **Networking → Services**.

TIP

If you switch from Inline Milvus to Remote Milvus, delete the existing pod to ensure the new environment variables and backing store are picked up cleanly.

3.8. INGESTING CONTENT INTO A LLAMA MODEL

You can quickly customize and prototype your retrievable content by ingesting raw text into your model from inside a Jupyter notebook. This approach voids requiring a separate ingestion pipeline. By using the LlamaStack SDK, you can embed and store text in your vector store in real-time, enabling immediate RAG workflows.

Prerequisites

- You have installed OpenShift 4.17 or newer.
- You have deployed a Llama 3.2 model with a vLLM model server and you have integrated LlamaStack.
- You have created a project workbench within a data science project.
- You have opened a Jupyter notebook and it is running in your workbench environment.
- You have installed the **llama_stack_client** version 0.2.22 or later in your workbench environment.
- You have a vector database identifier, or you plan to create or register one in this procedure.

- Your environment has network access to the vector database service through OpenShift.

Procedure

1. In a new notebook cell, install the **llama_stack_client** package and its dependencies:

```
%pip install llama_stack_client fire
```

2. In a new notebook cell, import RAGDocument and LlamaStackClient:

```
from llama_stack_client import RAGDocument, LlamaStackClient
```

3. In a new notebook cell, assign your deployment endpoint to the **base_url** parameter to create a LlamaStackClient instance:

```
client = LlamaStackClient(base_url="<your deployment endpoint>")
```

4. List the available models:

```
# Fetch all registered models
models = client.models.list()
```

5. Verify that the list of registered models includes your Llama model and an embedding model. Here is an example of a list of registered models:

```
[Model(identifier='llama-32-3b-instruct', metadata={}, api_model_type='llm', provider_id='vllm-inference', provider_resource_id='llama-32-3b-instruct', type='model', model_type='llm'),
 Model(identifier='ibm-granite/granite-embedding-125m-english', metadata={'embedding_dimension': 768.0}, api_model_type='embedding', provider_id='sentence-transformers', provider_resource_id='ibm-granite/granite-embedding-125m-english', type='model', model_type='embedding')]
```

6. Select the first LLM and the first embedding model:

```
model_id = next(m.identifier for m in models if m.model_type == "llm")

embedding_model = next(m for m in models if m.model_type == "embedding")
embedding_model_id = embedding_model.identifier
embedding_dimension = int(embedding_model.metadata["embedding_dimension"])
```

7. (Optional) Register a vector database (choose one). Skip if you already have a vector DB ID.

Example 3.1. Option 1: Inline Milvus Lite (embedded)

```
vector_db_id = "my_inline_db"

vector_store = client.vector_stores.create(
    name=vector_db_id,
    embedding_model=embedding_model_id,
    embedding_dimension=embedding_dimension,
    provider_id="milvus", # inline Milvus Lite
)
print(f"Registered inline Milvus Lite DB: {vector_db_id}")
```

**NOTE**

Use inline Milvus Lite for development and small datasets. Persistence and scale are limited compared to remote Milvus.

Example 3.2. Option 2: Remote Milvus (recommended for production)

```
vector_db_id = "my_remote_db"

vector_store = client.vector_stores.create(
    name=vector_db_id,
    embedding_model=embedding_model_id,
    embedding_dimension=embedding_dimension,
    provider_id="milvus-remote", # remote Milvus provider (v2.25+)
)
print(f"Registered remote Milvus DB: {vector_db_id}")
```

- Ensure your **LlamaStackDistribution** sets **MILVUS_ENDPOINT** (gRPC :19530) and **MILVUS_TOKEN**.
- Aside from the **provider_id**, ingestion and query APIs are identical for inline and remote Milvus.

1. If you already have a vector database, set its identifier:

```
# If a DB already exists, set it here instead of registering above
# Example:
# vector_db_id = "<your existing vector database ID>"
```

2. In a new notebook cell, define the raw text that you want to ingest into the vector store:

```
# Example raw text passage
raw_text = """
LlamaStack can embed raw text into a vector store for retrieval.
This example ingests a small passage for demonstration.
"""
```

3. In a new notebook cell, create a RAGDocument object to contain the raw text:

```
document = RAGDocument(
    document_id="raw_text_001",
    content=raw_text,
    mime_type="text/plain",
    metadata={"source": "example_passage"},
)
```

4. In a new notebook cell, ingest the raw text:

```
client.tool_runtime.rag_tool.insert(
    documents=[document],
    vector_db_id=vector_db_id,
```

```

        chunk_size_in_tokens=100,
    )
    print("Raw text ingested successfully")

```

5. In a new notebook cell, create a RAGDocument from an HTML source and ingest it into the vector store:

```

source = "https://www.paulgraham.com/greatwork.html"
print("rag_tool> Ingesting document:", source)

document = RAGDocument(
    document_id="document_1",
    content=source,
    mime_type="text/html",
    metadata={},
)

```

6. In a new notebook cell, ingest the content into the vector store:

```

client.tool_runtime.rag_tool.insert(
    documents=[document],
    vector_db_id=vector_db_id,
    chunk_size_in_tokens=50,
)
print("Raw text ingested successfully")

```

Verification

- Review the output to confirm successful ingestion. A typical response after ingestion includes the number of text chunks inserted and any warnings or errors.
- The model list returned by **client.models.list()** includes your Llama 3.2 model and an embedding model.

3.9. QUERYING INGESTED CONTENT IN A LLAMA MODEL

You can use the LlamaStack SDK in your Jupyter notebook to query ingested content by running retrieval-augmented generation (RAG) queries on raw text or HTML sources stored in your vector database. When you query the ingested content, you can perform one-off lookups or start multi-turn conversational flows without setting up a separate retrieval service.

Prerequisites

- You have installed OpenShift 4.17 or newer.
- You have enabled GPU support in OpenShift AI. This includes installing the Node Feature Discovery operator and NVIDIA GPU Operators. For more information, see [Installing the Node Feature Discovery operator](#) and [Enabling NVIDIA GPUs](#).
- If you are using GPU acceleration, you have at least one NVIDIA GPU available.
- You have logged in to OpenShift web console.
- You have activated the Llama Stack Operator in OpenShift AI.

- You have deployed an inference model, for example, the llama-3.2-3b-instruct model.
- You have configured a Llama Stack deployment by creating a **LlamaStackDistribution** instance to enable RAG functionality.
- You have created a project workbench within a data science project.
- You have opened a Jupyter notebook and it is running in your workbench environment.
- You have installed the **llama_stack_client** version 0.2.14 or later in your workbench environment.
- You have ingested content into your model.



NOTE

This procedure does not require any specific type of content. It only requires that you have already ingested some text, HTML, or document data into your vector database, and that this content is available for retrieval. If you have previously ingested content, that content will be available to query. If you have not ingested any content yet, the queries in this procedure will return empty results or errors.

Procedure

1. In a new notebook cell, install the **llama_stack** client package:

```
%pip install llama_stack_client
```

2. In a new notebook cell, import **Agent**, **AgentEventLogger**, and **LlamaStackClient**:

```
from llama_stack_client import Agent, AgentEventLogger, LlamaStackClient
```

3. In a new notebook cell, assign your deployment endpoint to the **base_url** parameter to create a **LlamaStackClient** instance. For example:

```
client = LlamaStackClient(base_url="http://lsd-llama-milvus-service:8321/")
```

4. In a new notebook cell, list the available models:

```
models = client.models.list()
```

5. Verify that the list of registered models includes your Llama model and an embedding model. Here is an example of a list of registered models:

```
[Model(identifier='llama-32-3b-instruct', metadata={}, api_model_type='llm', provider_id='vllm-inference', provider_resource_id='llama-32-3b-instruct', type='model', model_type='llm'),
 Model(identifier='ibm-granite/granite-embedding-125m-english', metadata={'embedding_dimension': 768.0}, api_model_type='embedding', provider_id='sentence-transformers', provider_resource_id='ibm-granite/granite-embedding-125m-english', type='model', model_type='embedding')]
```

6. Select the first LLM:

```
model_id = next(m.identifier for m in models if m.model_type == "llm")
```

7. If you have not already created a vector store, select an embedding model for registration in the next step:

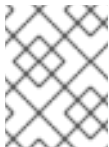
```
embedding = next(m for m in models if m.model_type == "embedding")
embedding_model_id = embedding.identifier
embedding_dimension = int(embedding.metadata["embedding_dimension"])
```

8. If you do not already have a vector store ID, register a vector store of your choice:

Example 3.3. Option 1: Inline Milvus Lite (embedded)

```
vector_db_id = "my_inline_db"

vector_store = client.vector_stores.create(
    name=vector_db_id,
    embedding_model=embedding_model_id,
    embedding_dimension=embedding_dimension,
    provider_id="milvus", # inline Milvus Lite
)
print(f"Registered inline Milvus Lite DB: {vector_db_id}")
```



NOTE

Use inline Milvus Lite for development and small datasets. Persistence and scale are limited compared to remote Milvus.

Example 3.4. Option 2: Remote Milvus (recommended for production)

```
vector_db_id = "my_remote_db"

vector_store = client.vector_stores.create(
    name=vector_db_id,
    embedding_model=embedding_model_id,
    embedding_dimension=embedding_dimension,
    provider_id="milvus-remote", # remote Milvus provider (v2.25+)
)
print(f"Registered remote Milvus DB: {vector_db_id}")
```

- Ensure your **LlamaStackDistribution** sets **MILVUS_ENDPOINT** (gRPC :19530) and **MILVUS_TOKEN**.
 - Aside from the **provide_id**, querying APIs are identical for inline and remote Milvus.
1. If you already have a vector database, set its identifier:

```
# If a DB already exists, set it here instead of registering above
# Example:
# vector_db_id = "<your existing vector database ID>"
```

2. In a new notebook cell, query the ingested content using the low-level RAG tool:

```
# Example RAG query for one-off lookups
query = "What benefits do the ingested passages provide for retrieval?"
result = client.tool_runtime.rag_tool.query(
    vector_db_ids=[vector_db_id],
    content=query,
)
print("Low-level query result:", result)
```

3. In a new notebook cell, query the ingested content by using the high-level Agent API:

```
# Create an Agent for conversational RAG queries
agent = Agent(
    client,
    model=model_id,
    instructions="You are a helpful assistant.",
    tools=[
        {
            "name": "builtin::rag/knowledge_search",
            "args": {"vector_db_ids": [vector_db_id]},
        }
    ],
)

prompt = "How do you do great work?"
print("Prompt>", prompt)

# Create a session and run a streaming turn
session_id = agent.create_session("rag_session")
response = agent.create_turn(
    messages=[{"role": "user", "content": prompt}],
    session_id=session_id,
    stream=True,
)

# Log and print the agent's response
for log in AgentEventLogger().log(response):
    log.print()
```

Verification

- The notebook prints query results for both the low-level RAG tool and the high-level Agent API.
- No errors appear in the output, confirming the model can retrieve and respond to ingested content.

3.10. PREPARING DOCUMENTS WITH DOCLING FOR LLAMA STACK RETRIEVAL

You can transform your source documents with a Docling-enabled data science pipeline and ingest the output into a Llama Stack vector store by using the Llama Stack SDK. This modular approach separates document preparation from ingestion, yet still delivers an end-to-end, retrieval-augmented generation (RAG) workflow.

The pipeline registers a Milvus vector database and downloads the source PDFs, then splits them for parallel processing and converts each batch to Markdown with Docling. It generates sentence-transformer embeddings from the Markdown and stores them in the vector store, making the documents instantly searchable in Llama Stack.

Prerequisites

- You have installed OpenShift 4.17 or newer.
- You have enabled GPU support in OpenShift AI. This includes installing the Node Feature Discovery operator and NVIDIA GPU Operators. For more information, see [Installing the Node Feature Discovery operator](#) and [Enabling NVIDIA GPUs](#).
- You have logged in to OpenShift web console.
- You have a data science project and access to pipelines in the OpenShift AI dashboard.
- You have created and configured a pipeline server within the data science project that contains your workbench.
- You have activated the Llama Stack Operator in OpenShift AI.
- You have deployed an inference model, for example, the llama-3.2-3b-instruct model.
- You have configured a Llama Stack deployment by creating a **LlamaStackDistribution** instance to enable RAG functionality.
- You have created a project workbench within a data science project.
- You have opened a Jupyter notebook and it is running in your workbench environment.
- You have installed the **llama_stack_client** version 0.2.14 or later in your workbench environment.
- You have installed local object storage buckets and created connections, as described in [Adding a connection to your data science project](#).
- You have compiled to YAML a data science pipeline that includes a Docling transform, either one of the RAG demo samples or your own custom pipeline.
- Your data science project quota allows between 500 millicores (0.5 CPU) and 4 CPU cores for the pipeline run.
- Your data science project quota allows from 2 GiB up to 6 GiB of RAM for the pipeline run.
- If you are using GPU acceleration, you have at least one NVIDIA GPU available.

Procedure

1. In a new notebook cell, install the **llama_stack** client package:

```
%pip install llama_stack_client
```

2. In a new notebook cell, import Agent, AgentEventLogger, and LlamaStackClient:

```
from llama_stack_client import Agent, AgentEventLogger, LlamaStackClient
```

3. In a new notebook cell, assign your deployment endpoint to the **base_url** parameter to create a LlamaStackClient instance:

```
client = LlamaStackClient(base_url="<your deployment endpoint>")
```

4. List the available models:

```
models = client.models.list()
```

5. Select the first LLM and the first embedding model:

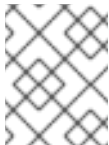
```
model_id = next(m.identifier for m in models if m.model_type == "llm")
embedding_model = next(m for m in models if m.model_type == "embedding")
embedding_model_id = embedding_model.identifier
embedding_dimension = embedding_model.metadata["embedding_dimension"]
```

6. In a new notebook cell, register a vector database (choose one option):

Example 3.5. Option 1: Inline Milvus Lite (embedded)

```
vector_db_id = "my_inline_db"

vector_store = client.vector_stores.create(
    name=vector_db_id,
    embedding_model=embedding_model_id,
    embedding_dimension=embedding_dimension,
    provider_id="milvus", # inline Milvus Lite
)
print(f"Registered inline Milvus Lite DB: {vector_db_id}")
```



NOTE

Inline Milvus Lite is best for development. Data durability and scale are limited compared to remote Milvus.

Example 3.6. Option 2: Remote Milvus (recommended for production)

```
vector_db_id = "my_remote_db"

vector_store = client.vector_stores.create(
    name=vector_db_id,
    embedding_model=embedding_model_id,
    embedding_dimension=embedding_dimension,
    provider_id="milvus-remote", # remote Milvus provider (v2.25+)
)
print(f"Registered remote Milvus DB: {vector_db_id}")
```

- Ensure your **LlamaStackDistribution** includes **MILVUS_ENDPOINT** and **MILVUS_TOKEN**.
- Aside from the **provider_id**, ingestion and query APIs are identical between inline and remote Milvus.

+



IMPORTANT

If you are using the sample Docling pipeline from the RAG demo repository, the pipeline registers the database automatically and you can skip this step. However, if you are using your own pipeline, you must register the database yourself.

1. In the OpenShift web console, import the YAML file containing your docling pipeline into your data science project, as described in [Importing a data science pipeline](#).
2. Create a pipeline run to execute your Docling pipeline, as described in [Executing a pipeline run](#). The pipeline run inserts your PDF documents into the vector database. If you run the Docling pipeline from the [RAG demo samples repository](#), you can optionally customize the following parameters before starting the pipeline run:
 - **base_url**: The base URL to fetch PDF files from.
 - **pdf_filenames**: A comma-separated list of PDF filenames to download and convert.
 - **num_workers**: The number of parallel workers.
 - **vector_db_id**: The Milvus vector database ID.
 - **service_url**: The Milvus service URL.
 - **embed_model_id**: The embedding model to use.
 - **max_tokens**: The maximum tokens for each chunk.
 - **use_gpu**: Enable or disable GPU acceleration.

Verification

1. In your Jupyter notebook, query the LLM with a question that relates to the ingested content. For example:

```
from llama_stack_client import Agent, AgentEventLogger
import uuid

rag_agent = Agent(
    client,
    model=model_id,
    instructions="You are a helpful assistant",
    tools=[
        {
            "name": "builtin::rag/knowledge_search",
            "args": {"vector_db_ids": [vector_db_id]},
        }
    ],
)

prompt = "What can you tell me about the birth of word processing?"
print("prompt>", prompt)
```

```

session_id = rag_agent.create_session(session_name=f"s{uuid.uuid4().hex}")

response = rag_agent.create_turn(
    messages=[{"role": "user", "content": prompt}],
    session_id=session_id,
    stream=True,
)

for log in AgentEventLogger().log(response):
    log.print()

```

2. Query chunks from the vector database:

```

query_result = client.vector_io.query(
    vector_db_id=vector_db_id,
    query="what do you know about?",
)
print(query_result)

```

3.11. ABOUT LLAMA STACK SEARCH TYPES

Llama Stack supports keyword, vector, and hybrid search modes for retrieving context in retrieval-augmented generation (RAG) workloads. Each mode offers different tradeoffs in precision, recall, semantic depth, and computational cost.

3.11.1. Supported search modes

3.11.1.1. Keyword search

Keyword search applies lexical matching techniques, such as TF-IDF or BM25, to locate documents that contain exact or near-exact query terms. This approach is effective when precise term-matching is critical and remains widely used in information-retrieval systems. For more information, see [The Probabilistic Relevance Framework: BM25 and Beyond](#).

3.11.1.2. Vector search

Vector search encodes documents and queries as dense numerical vectors, known as embeddings, and measures similarity with metrics such as cosine similarity or inner product. This approach captures contextual meaning and supports semantic matching beyond exact word overlap. For more information, see [Billion-scale similarity search with GPUs](#).

3.11.1.3. Hybrid search

Hybrid search blends keyword and vector techniques, typically by combining individual scores with a weighted sum or methods, such as Reciprocal Rank Fusion (RRF). This approach returns results that balance exact matches with semantic relevance. For more information, see [Sparse, Dense, and Hybrid Retrieval for Answer Ranking](#).

3.11.2. Retrieval database support

Milvus is the supported retrieval database for Llama Stack. It currently provides vector search. However, keyword and hybrid search capabilities are not currently supported.

CHAPTER 4. BENCHMARKING EMBEDDING MODELS WITH BEIR DATASETS AND LLAMA STACK

This procedure explains how to set up, run, and verify embedding-model benchmarks by using the Llama Stack framework. Embedding models are neural networks that convert text or other data into dense numerical vectors, called embeddings, which capture semantic meaning. In retrieval-augmented generation (RAG) systems, embeddings enable semantic search so that the system retrieves the documents most relevant to a query.

Selecting an embedding model depends on several factors, such as the content type, accuracy requirements, performance needs, and model license. The **beir_benchmarks.py** script compares the retrieval accuracy of embedding models by using standardized information-retrieval benchmarks from the BEIR framework. The script is included in the [RAG](#) repository, which provides demonstrations, benchmarking scripts, and deployment guides for the RAG Stack on OpenShift.

The examples use the **sentence-transformers** inference provider, which you can replace with another provider if required.

Prerequisites

- You have cloned the <https://github.com/opendatahub-io/rag> repository.
- You have changed into the **/rag/benchmarks/beir-benchmarks** directory.
- You have initialized and activated a virtual environment.
- You have defined and installed the relevant script package dependencies to a **requirements.txt** file.
- You have built the Llama Stack starter distribution to install all dependencies.
- You have verified that your vector database is accessible and configured in the **run.yaml** file, and that any required embedding models were preloaded or registered with Llama Stack.

NOTE

The default supported embedding models are **granite-embedding-30m** and **granite-embedding-125m**, served by the **sentence-transformers** framework. Ollama is not required for basic benchmarks but can be used to serve custom embedding models.

To register an additional embedding model, such as **all-MiniLM-L6-v2**, perform the following steps:

1. Start the Llama Stack server:

```
MILVUS_URL=milvus uv run llama stack run run.yaml
```

2. Register the model by using the Llama Stack client. For example:

```
llama-stack-client models register all-MiniLM-L6-v2 \
  --provider-id sentence-transformers \
  --provider-model-id all-minilm:latest \
  --metadata {"embedding_dimension": 384} \
  --model-type embedding
```

- You have shut down the Llama Stack server before running the benchmark script.

Procedure

1. Run the **beir_benchmarks.py** benchmarking script:

- Enter the following command to use the configuration from **run.yaml** and the default dataset (**scifact**):

```
MILVUS_URL=milvus uv run python beir_benchmarks.py
```

- Alternatively, enter the following command to connect to a custom Llama Stack server:

```
LLAMA_STACK_URL="http://localhost:8321" MILVUS_URL=milvus uv run python  
beir_benchmarks.py
```

2. Use environment variables and command-line options to modify the benchmark run. For example, set the environment variable **ENABLE_MILVUS=milvus** before executing the script.

- Enter the following command to benchmark with a specific LLM by using default settings:

```
ENABLE_MILVUS=milvus uv run python beir_benchmarks.py
```

- Enter the following command to use a larger batch size for document ingestion:

```
ENABLE_MILVUS=milvus uv run python beir_benchmarks.py --batch-size 300
```

- Enter the following command to benchmark multiple datasets (for example, **scifact** and **scidocs**):

```
ENABLE_MILVUS=milvus uv run python beir_benchmarks.py \  
--dataset-names scifact scidocs
```

- Enter the following command to compare embedding models (for example, **granite-embedding-30m** and **all-MiniLM-L6-v2**):

```
ENABLE_MILVUS=milvus uv run python beir_benchmarks.py \  
--embedding-models granite-embedding-30m all-MiniLM-L6-v2
```

- Enter the following command to use a custom BEIR-compatible dataset:

```
ENABLE_MILVUS=milvus uv run python beir_benchmarks.py \  
--dataset-names my-dataset \  
--custom-datasets-urls https://example.com/my-beir-dataset.zip
```

- Enter the following command to change the vector database provider. The following example changes the vector database provider to remote Milvus:

```
ENABLE_MILVUS=milvus uv run python beir_benchmarks.py \  
--vector-db-provider-id remote-milvus
```

Command-line options

- **--vector-db-provider-id**

- **Description:** Specifies the vector database provider to use.
- **Type:** String.
- **Default:** **milvus**.
- **Example:**

```
--vector-db-provider-id remote-milvus
```

- **--dataset-names**

- **Description:** Specifies which BEIR datasets to use for benchmarking. Use this option together with **--custom-datasets-urls** when testing custom datasets.
- **Type:** List of strings.
- **Default:** **["scifact"]**.
- **Example:**

```
--dataset-names scifact scidocs nq
```

- **--embedding-models**

- **Description:** Specifies the embedding models to compare. Models must be defined in the **run.yaml** file.
- **Type:** List of strings.
- **Default:** **["granite-embedding-30m", "granite-embedding-125m"]**.
- **Example:**

```
--embedding-models all-MiniLM-L6-v2 granite-embedding-125m
```

- **--batch-size**

- **Description:** Controls how many documents are processed per batch during ingestion. Larger batch sizes improve speed but use more memory.
- **Type:** Integer.
- **Default:** **150**.
- **Example:**

```
--batch-size 50  
--batch-size 300
```

- **--custom-datasets-urls**

- **Description:** Specifies URLs for custom BEIR-compatible datasets. Use this option with **--dataset-names**.
- **Type:** List of strings.
- **Default:** [].
- **Example:**

```
--dataset-names my-custom-dataset \
--custom-datasets-urls https://example.com/my-dataset.zip
```



NOTE

Custom BEIR datasets must follow the required file structure and format:

```
dataset-name.zip/
├── qrels/
│   └── test.tsv    # Maps query IDs to document IDs with relevance scores
├── corpus.jsonl    # Document collection with document IDs, titles, and text
└── queries.jsonl   # Test queries with query IDs and question text
```

Verification

To verify that the benchmark completed successfully and to review the results, perform the following steps:

1. Locate the **results** directory. All output files are saved to the following path:
<path-to>/rag/benchmarks/embedding-models-with-beir/results
2. Examine the output. Compare your results with the sample output structure. The report includes performance metrics such as **map@cut_k** and **ndcg@cut_k** for each dataset and embedding model pair. The script also calculates a statistical significance test (**p-value**).

Example output (for *scifact* and *map_cut_10*):

```
scifact map_cut_10
granite-embedding-125m : 0.6879
granite-embedding-30m  : 0.6578
p_value                 : 0.0150
```

p_value < 0.05 indicates a statistically significant difference.

The granite-embedding-125m model performs better for this dataset and metric.

3. Interpret the results. A **p-value** below **0.05** indicates that the performance difference between models is statistically significant. Use these results to identify which embedding model performs best for your dataset.